

Questões de prova seminário Transformers Limitations

Questão 1

Por que o Transformer, mesmo sendo tão poderoso, não consegue lidar bem com linguagens como PARITY (verifica se a quantidade de 1s em uma sentença é par ou ímpar) e 2DYCK (verifica se os parênteses estão corretamente balanceados)?

Uma boa resposta para essa pergunta precisa falar de dois conceitos principais: primeiro, o funcionamento do self-attention que é global, mas não sequencial e depois, o tipo de memória e estrutura hierárquica que as linguagens PARITY e 2DYCK exigem. Também é importante relacionar isso com o argumento teórico do Hahn sobre escalabilidade e limitação estrutural.

Resposta:

Porque o mecanismo de self-attention não tem memória infinita. O modelo processa toda a sequência ao mesmo tempo, sem uma pilha de memória que permita lembrar a ordem e a profundidade das informações. Por isso, ele não consegue contar ou representar hierarquias recursivas quando a entrada cresce.

Em PARITY, isso significa que o Transformer perde a noção de alternância entre “par” e “ímpar” em sequências longas. Em 2DYCK, ele não consegue acompanhar corretamente os parênteses aninhados, pois ambos os casos exigem uma memória tipo pilha. Só seria possível resolver essas tarefas se o número de camadas ou cabeças aumentasse junto com o tamanho da entrada, o que torna o modelo não escalável.

Questão 2

Com base no artigo “*On Limitations of the Transformer Architecture*” (Peng, Narayanan e Papadimitriou, 2024), explique por que o mecanismo de atenção do Transformer apresenta limitações fundamentais em tarefas de **composição de funções** e como essas limitações se relacionam com a ocorrência de **alucinações** em modelos de linguagem de larga escala (LLMs).

Resposta:

O artigo demonstra que o Transformer, devido à sua estrutura matemática baseada no mecanismo de atenção, possui limitações teóricas na realização de **composição de funções** — operações em que o resultado de uma função serve de entrada para outra, como responder “Qual é a profissão da mãe de João?”.

Essas limitações decorrem de restrições impostas pela **complexidade da comunicação** entre as partes do modelo. Formalmente, a prova apresentada mostra que um Transformer de uma única camada com H cabeças de atenção, dimensão de embedding d e precisão p , não consegue resolver corretamente a composição de funções quando $H(d+1)p < n \log n$, sendo n o tamanho do domínio do problema. Isso ocorre porque a arquitetura não consegue transmitir informação suficiente entre suas partes — uma forma de **gargalo informacional**.

A origem dessa limitação está no uso do **softmax**, que calcula pesos de atenção a partir de similaridades locais entre tokens, o que restringe a capacidade de capturar relações não locais e hierárquicas necessárias para composições complexas.

Essas restrições explicam parte do fenômeno das **alucinações** em LLMs: quando o modelo tenta compor informações de forma relacional (por exemplo, inferir parentescos ou encadeamentos lógicos), a carência de capacidade de composição causa respostas inconsistentes, mesmo quando todos os fatos relevantes estão no contexto. Assim, as alucinações são interpretadas como manifestações empíricas de limites formais da arquitetura Transformer.

Questão (3)

No paper "Representational Strengths and Limitations of Transformers", qual o papel da dimensão do *embedding*, m ?

Resposta:

A dimensão do embedding m é o principal fator que determina a capacidade representacional de uma camada de autoatenção. Ela define o espaço vetorial onde são projetadas as chaves, consultas e valores, limitando assim a quantidade de informação e o número de relações entre elementos da sequência que o modelo pode codificar.

Os autores demonstram que, no problema de *q-sparse averaging* (qSA), uma unidade de self-attention só pode aproximar corretamente a tarefa se m crescer pelo menos linearmente com q — ou seja, $m \geq q$. Isso mostra que, quanto mais elementos precisam ser combinados, maior deve ser o embedding para representar as interações necessárias. Contudo, q cresce de forma esparsa em relação ao tamanho total da entrada N , assim a dependência prática é $m \geq q \log(N)$.

Já em tarefas de correspondência, como *Match2* e *Match3*, observa-se que a primeira pode ser resolvida com embedding pequeno, enquanto a segunda exige que m (ou o número de cabeças) cresça polinomialmente com N .

Portanto, m atua como canal de comunicação e limitador de complexidade: embeddings maiores aumentam a expressividade, mas a custo de maior complexidade computacional.

Questão (4)

Por que o transformer não consegue resolver eficientemente o Match3(X)?

Resposta:

O Transformer não consegue resolver eficientemente o problema Match3(X) porque essa tarefa exige interações de ordem superior, isto é, relações entre três elementos simultaneamente. O mecanismo de autoatenção padrão, no entanto, é intrinsecamente par-a-par (pairwise). Cada cabeça de atenção calcula escores por meio de produtos internos entre pares de embeddings — as chaves e consultas —, o que permite identificar padrões binários, como similaridades ou correspondências entre dois tokens, mas não combinações conjuntas entre três ou mais.

No caso de Match3, o modelo precisa detectar triplas (x_i, x_{j1}, x_{j2}) que satisfaçam uma relação conjunta, algo que não pode ser decomposto em somas de interações independentes entre pares. Por isso, os autores provam que, para resolver Match3 de forma exata, um Transformer comum precisaria que a dimensão do embedding m , o número de cabeças H ou a profundidade da rede crescessem polinomialmente com o tamanho da sequência N , o que o torna ineficiente.

Sendo assim, essa limitação não decorre de falta de aprendizado, mas de estrutura: a arquitetura do transformer restringe o fluxo de informação a interações bilineares. Portanto, resolver Match3 de modo eficiente exigiria mecanismos de atenção de ordem superior, capazes de representar explicitamente as dependências triádicas, como proposto pelos autores com a third-order self-attention.

Questão 5

Com base no artigo “Transformers Learn Shortcuts to Automata” (ICLR 2023), os autores mostram que os Transformers podem aprender “shortcuts”, isto é, atalhos computacionais que reproduzem o resultado correto sem seguir o processo sequencial e lógico para a resolução do problema. Esses atalhos apesar conseguirem uma melhor complexidade para a resolução de semiautômatos, mostram uma grande limitação da arquitetura transformer. Discorra sobre como essa limitação afeta a capacidade dos Transformers em casos fora da distribuição de treino.

Resposta:

Os shortcuts limitam a generalização dos Transformers porque fazem o modelo aprender associações diretas entre entrada e saída, sem reconstruir o processo sequencial que leva ao resultado. Assim, ele “pula etapas” e resolve o problema de forma superficial, baseando-se em padrões estatísticos do treino, sem compreender o contexto subjacente.

Quando enfrenta sequências mais longas ou distribuições diferentes, o modelo falha, pois seus atalhos não capturam a lógica da troca de estados e a coerência temporal. Ele não generaliza, apenas repete padrões aprendidos.

Como o mecanismo de atenção não reproduz a recorrência necessária para entender dependências temporais complexas, os shortcuts tornam o cálculo eficiente, mas o raciocínio frágil e limitado. O Transformer acerta o resultado, mas não compreende o processo de forma profunda ou estrutural.